Solution: Max Consecutive Ones III

Problem Statement

Given an array `nums` containing `1` and `0` digits, return the maximum number of consecutive `1`s if you can flip at most any `k` `0`s.

Examples

- **Example 1**:
  - **Input**: nums = `[1, 0, 1, 1, 0, 0, 1, 1, 1, 0]`, k = `2`
  - **Expected Output**: `7`
  - **Justification**: Flipping the two 0s in the middle section `[0, 0]` results in the longest continuous sequence of 1s: `[1, 1, 1, 1, 1, 1, 1]`.
- **Example 2**:
  - **Input**: nums = `[0, 1, 1, 0, 1, 0, 1, 0, 1]`, k = `1`
  - **Expected Output**: `4`
  - **Justification**: Flipping the `0` at index 3 will yield a maximum continuous sequence of four 1s.
- **Example 3**:
  - **Input**: nums = `[1, 1, 0, 0, 0, 1, 1, 1, 0, 1, 1]`, k = `3`
  - **Expected Output: `8`
  - **Justification**: Flipping the three 0s in the middle `[0, 0, 0]` results in the longest sequence of 1s: `[1, 1, 1, 1, 1, 1, 1, 1]`.

Solution

To solve this problem, we can use a sliding window approach. This technique involves expanding and contracting a window within the array to encapsulate sequences that fit our criteria. By moving this window throughout the array, we can identify the longest sequence of 1s that can be obtained by flipping at most a given number of 0s. The reason this method is effective is that it allows us to efficiently examine each potential sequence without redundant calculations. This approach is not only intuitive but also optimizes the process by reducing unnecessary computations.

The key lies in maintaining the count of zeros within our current window. We expand the window by moving the right pointer forward if the number of zeros within it is less than or equal to the allowed limit. If the number of zeros exceeds the limit, we contract the window by moving the left pointer forward. Throughout this process, we keep track of the maximum window size, which ultimately gives us the longest sequence of 1s achievable.

Step-by-step Algorithm

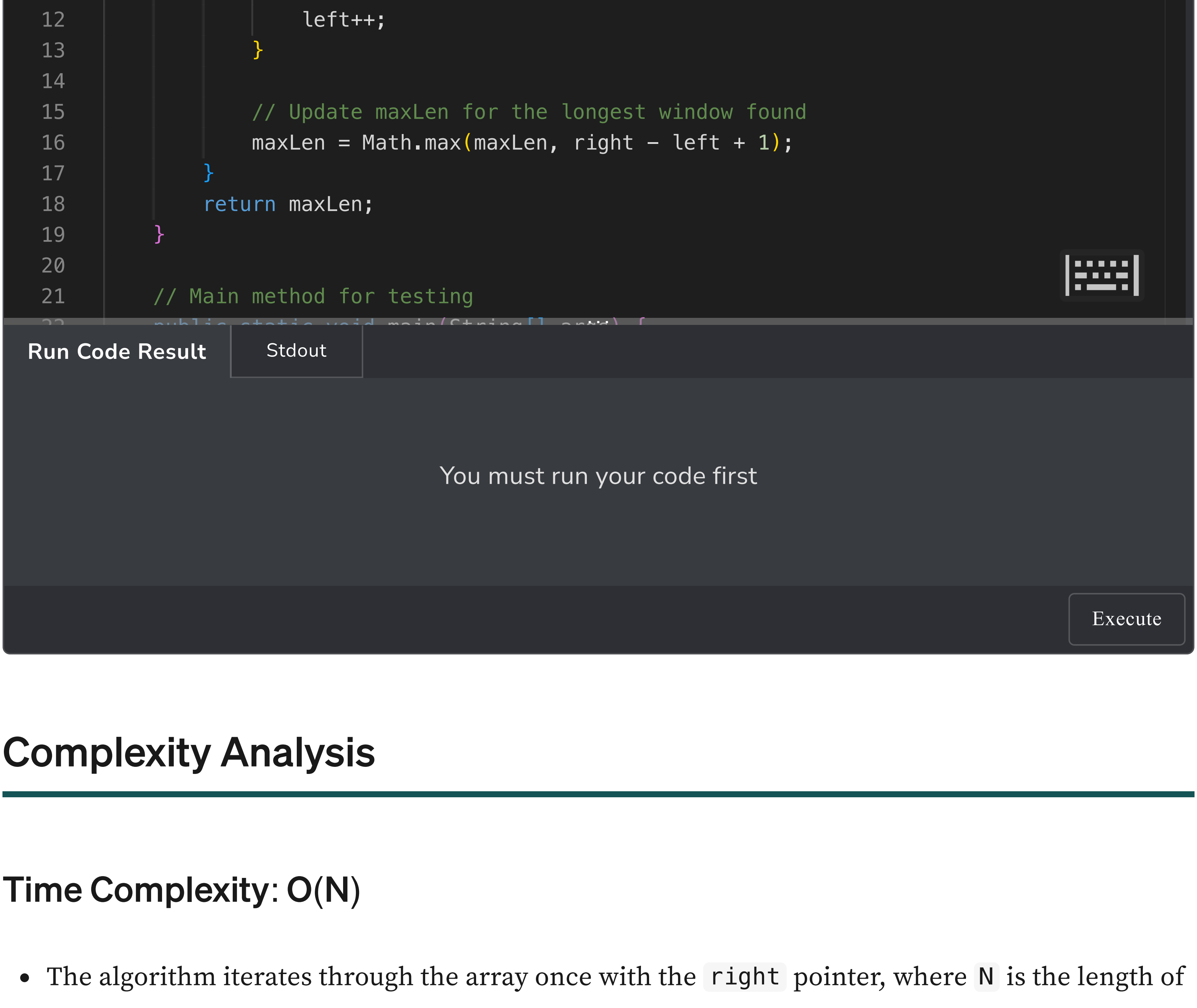
- Initialize two pointers, `left` and `right`, both set to 0, and a variable `maxLen` for tracking the maximum length of 1s sequence.
- Initialize a variable `zeroCount` to keep track of the number of zeros in the current window.
- Iterate through the array using the `right` pointer.
  - If the element at the `right` pointer is 0, increment `zeroCount`.
  - While `zeroCount` is greater than the allowed number of flips, move the `left` pointer forward, decrementing `zeroCount` if a 0 is left behind.
  - Update `maxLen` with the maximum of its current value and the size of the current window (`right - left + 1`).
- Return `maxLen` as the result.

Algorithm Walkthrough

Let's consider the input: `[1, 1, 0, 0, 0, 1, 1, 1, 0, 1, 1]`, `K = 3`).

1. **Initial State**:
   - `left = 0`, `right = 0`, `zeroCount = 0`, `maxLen = 0`.
2. **First step**:
   - `A[0] = 1`, no change in `zeroCount`.
   - `right = 0`, `maxLen = 1` (current window: `[1]`).
3. **Move Right to 1**:
   - `A[1] = 1`, no change in `zeroCount`.
   - `right = 1`, `maxLen = 2` (current window: `[1, 1]`).
4. **Move Right to 2**:
   - `A[2] = 0`, `zeroCount = 1 <= k (3)`.
   - `right = 2`, `maxLen = 3` (current window: `[1, 1, 0]`).
5. **Move Right to 3**:
   - `A[3] = 0`, `zeroCount = 2 <= k (3)`.
   - `right = 3`, `maxLen = 4` (current window: `[1, 1, 0, 0]`).
6. **Move Right to 4**:
   - `A[4] = 0`, `zeroCount = 3 <= k (3)`.
   - `right = 4`, `maxLen = 5` (current window: `[1, 1, 0, 0, 0]`).
7. **Move Right to 5**:
   - `A[5] = 1`, `zeroCount = 3 <= k (3)`.
   - `right = 5`, `maxLen = 6` (current window: `[1, 1, 0, 0, 0, 1]`).
8. **Move Right to 6**:
   - `A[6] = 1`, `zeroCount = 3 <= k (3)`.
   - `right = 6`, `maxLen = 7` (current window: `[1, 1, 0, 0, 0, 1, 1]`).
9. **Move Right to 7**:
   - `A[7] = 1`, `zeroCount = 3 <= k (3)`.
   - `right = 7`, `maxLen = 8` (current window: `[1, 1, 0, 0, 0, 1, 1, 1]`).
10. **Move Right to 8 and Adjust Left**:
    - `A[8] = 0`, `zeroCount = 4` > `K`.
    - Shift `left` to 1 (`A[1] = 1`, no change in `zeroCount`).
    - `right = 8`, `maxLen = 8` (current window: `[1, 0, 0, 0, 1, 1, 1, 0]`).
11. **Continue Moving Right and Adjusting Left**:
    - Continue this process, moving `right` forward and adjusting `left` when `zeroCount` exceeds `K`.
    - Update `maxLen` each time.
12. **Final Output**:
    - The loop ends when `right` reaches the end of the array.
    - Final `maxLen` is determined (expected to be `8` for this example).

Code



```java
public class Solution {
    public int longestOnes(int[] A, int K) {
        int left = 0, maxLen = 0, zeroCount = 0;
        // Iterate through the array with right pointer
        for (int right = 0; right < A.length; right++) {
            // Increment zeroCount if current element is 0
            if (A[right] == 0) zeroCount++;

            // Move left pointer to reduce zeroCount within limit
            while (zeroCount > K) {
                if (A[left] == 0) zeroCount--;
                left++;
            }

            // Update maxLen for the longest window found
            maxLen = Math.max(maxLen, right - left + 1);
        }
        return maxLen;
    }

    // Main method for testing
```

```
You must run your code first
```

Complexity Analysis

Time Complexity: O(N)

- The algorithm iterates through the array once with the `right` pointer, where `N` is the length of the array.
- The `left` pointer moves only in one direction and can move at most `N` times throughout the entire array.
- Since both pointers traverse the array at most once, the overall time complexity is linear, `O(N)`.

Space Complexity: O(1)

- The space used by the algorithm is independent of the size of the input array.
- Only a fixed number of variables (`left`, `right`, `maxLen`, `zeroCount`) are used, which occupy constant space.
- Thus, the space complexity is constant, O(1).

← Previous
Max Consecutive Ones III (Medium)

Next →
Subarray Sum Equals K (Medium)

☐ Mark as Completed